

Interpretable Monocular Depth Estimation Using Patch-Based Polynomial Regression

Arnav Kamath¹, Dhruva Karkada²

¹Foothill High School ²University of California, Berkeley

Abstract

Estimating depth from a single image is a difficult task with many applications, especially in robotics. Most current methods rely on large deep learning models trained on enormous datasets, but these approaches are resource intensive and lack transparency in how predictions are made. In this work, we present a regression-based method for monocular depth estimation. The model works by extracting simple features from small patches of the image and using polynomial regression to predict depth values. The predictions are then refined using semantic segmentation to reduce noise and improve consistency. We evaluated the approach on a well-known indoor dataset and found that it produced reasonably accurate results while remaining interpretable and requiring minimal compute power.

Introduction

Understanding depth is an important part of how humans and machines interact with the physical world. Whether it is a drone avoiding obstacles or a robot trying to pick up an object, they both rely on depth information to make accurate decisions. Without it, it becomes difficult to understand the environment: different sized objects can appear similar sized and relative distances cannot be estimated accurately.

Traditional approaches to depth estimation often depend on multiple viewpoints or specialized hardware. Stereo vision, for example, uses disparities between two cameras and applies triangulation to calculate depth.¹ Another common approach uses LiDAR technology to estimate depth.² However, in many cases, such methods are constrained by space or resource limitations. There might only be space for one camera, or the system might not have enough computational

power to support advanced hardware. Monocular depth estimation addresses these limitations by estimating depth from a single image. This presents an important question. How can we infer 3D features from 2D images?

Most recent work in monocular depth estimation has focused on large neural networks trained on massive datasets.^{3,4} While these models can be highly precise, they are complex, resource heavy, and do not clearly explain how predictions are made. Our work takes a different approach by using regression, a simpler mathematical method that directly links image features to depth. Unlike deep learning models, regression allows us to analyze the role of each feature separately, making the reasoning process clearer. This approach aligns with our goal to balance performance with simplicity and highlight *why* the model predicts certain depth patterns rather than only *what* it predicts.

Methods

Our method for monocular depth estimation is based on a patch-level regression model. Instead of processing full images directly, we divided each image into small patches, calculated a set of features for each patch, and used these features to predict the depth of that patch. The true depth value of each patch was computed by taking the average of the depth values for each pixel in the patch, which gave us a single target value for training. This reduced the complexity of the process and helped the model focus on depth cues at the local level rather than across the full image. We trained and tested this model on the NYU Depth V2 dataset, an established dataset that contains a variety of indoor scenes.⁵ The overall process can be divided into three parts: feature extraction, regression model training, and depth map generation. A more detailed list of steps is shown in Figure 1.

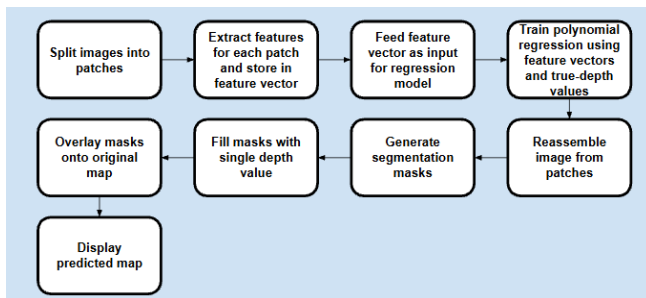


Figure 1. An overview of the steps used in this method.

1. Feature Extraction

Each image was divided into non-overlapping 8x8 patches. For each patch, we extracted nine features chosen to capture different elements related to depth:

- 1) Red Channel Mean: Average intensity of the red channel in the patch. This provides

color information that could be linked to materials or lighting.

- 2) Green Channel Mean: Average intensity of the green channel in the patch, complementing the red channel.
- 3) Blue Channel Mean: Average intensity of the blue channel in the patch, completing the RGB information.
- 4) Row Position: The vertical position of the patch, normalized by the height of the image. This helps the model understand perspective since patches lower in the image are often closer.
- 5) Column Position: The horizontal position of the patch, normalized by the width of the image. This also helps the model understand perspective.
- 6) Distance from Image Center: The Euclidean distance from the center of the patch to the center of the image, normalized by the image diagonal. This provides geometric information for the model.
- 7) Focal Length: A value based on the patch's position that approximates the focal region of the camera.
- 8) Gradient Magnitude: Average edge strength, computed using Sobel filters.⁶ This helps the model interpret textures since edges and textures tend to be stronger for nearby objects.
- 9) Patch Brightness: Mean grayscale intensity of the patch. Helps the model understand illumination and shading.

The information acquired from these features is stored in a feature vector which is fed as an input into the regression model. Since these features are all calculated directly from the image pixels, it is clear what exactly the model is using to generate

its predictions. Figure 2 shows the outputs of these features.

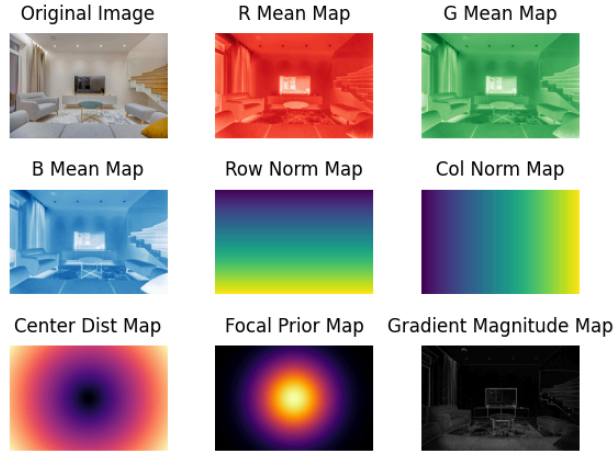


Figure 2. A visual representation of the features above calculated on a sample image.⁷ These features are then provided as input for the regression model.

2. Regression Model Training

After iterating through the image and calculating the features from each patch, we trained a polynomial regression model to predict depth values from the feature vectors.

Preprocessing

Before training, we standardized all features using a StandardScaler.⁸ This normalized all features between 0 and 1, and made sure that no single feature dominated the regression due to scale differences. We also applied a log transformation to the true depth values to reduce the effects of large outliers.

Training Procedure

We trained the regression model on a subset of the NYU Depth V2 dataset.⁵ The subset was divided into training and test splits, with approximately 80% of images used for training and about 20% used for testing. We used a second-degree polynomial expansion, which was able to identify

a non-linear relationship between the features and corresponding depth measurements.

Prediction Transformation

Since the predicted depth values were generated in log-space during training, we converted them back to linear depth values by exponentiating them. This made sure that the predicted values matched the scale of the true depth values.

3. Depth Map Generation

After obtaining the predictions for each patch from the regression model, we generated the full-image depth maps through a three-step process: image reconstruction, segmentation mask overlay, and visualization.

Image Reconstruction

To make sure that predictions were uniform across each patch, we broadcasted the predicted depth value for each patch over the entire patch. We then reconstructed the image by putting the patches back together in their original orientation.

Segmentation Mask Overlay

The patch-based approach presented a problem, however. The predicted depth maps looked patchy and inconsistent, even within a single object. To solve this, we used semantic segmentation to identify objects and make every pixel within each object have a constant value. We used a pretrained Mask R-CNN for this task, implemented through the torchvision library.⁹ This CNN was perfect for this task as it outputs binary masks per object in the image, making it easy to evaluate whether a pixel is within an object or not.⁹ First, we generated all masks contained within the image, along with their respective certainty scores. Masks with a certainty greater than 50% were kept while

masks with a certainty less than 50% were discarded. Using these masks, we adjusted the predicted depth map so that all pixels inside an object (inside the object mask) were assigned the closest depth value predicted for that object. This made it so that every object had a single, consistent prediction. Since the object masks were opaque, we overlaid the masks onto the depth map. We used a 90% overlay for the original map and a 10% overlay for the segmentation masks to achieve a good balance.

Visualization

To display results, we outputted the original RGB image, the true depth map, and predicted depth map side by side. We used a continuous colormap where cooler colors represented closer regions and warmer colors represented farther ones.

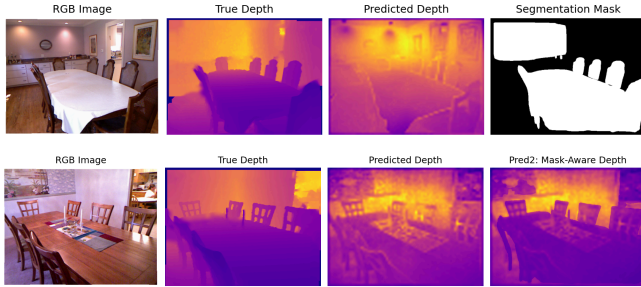


Figure 3. Predicted depth maps for two dining room settings at different stages in model development. The first row shows an early stage prediction by the regression model as well as the masks generated by the Mask R-CNN in binary form.⁹ The second row shows a later prediction, and compares the original prediction with the prediction overlaid with the segmentation mask map.

As seen in Figure 3, incorporating semantic segmentation significantly improved the predictions, and made them more consistent across objects.

Results and Discussion

The regression-based monocular depth estimation approach produced depth maps that were both interpretable and reasonably accurate. On test images from the NYU Depth V2 dataset, the model successfully predicted relative depth, especially in structured indoor scenes.⁵ Quantitatively, the model achieved an average mean squared error of approximately 0.05 m² over the testing set.

While the regression alone produced depth maps with the correct overall structure, there were occasional inconsistencies, especially around object boundaries and textured regions. To address these issues, the semantic segmentation overlay was introduced. This adjustment was effective and it helped remove noise within uniform regions, such as objects. At the same time, the weighted overlay step ensured that the segmentation did not overpower the regression predictions, preserving finer details as well.

The nine features all played an important role, but each contributed differently to the overall performance. Mean color values provided lighting information, spatial features gave the model a sense of perspective, and the texture features helped the model analyze edges and shadows. The regression coefficients, displayed in Table 1, show the relative contribution of each of the features.

Feature	Regression Coefficient
Red Channel Mean	0.1114
Green Channel Mean	-0.2910
Blue Channel Mean	0.1723
Row Position	-20856449201.0965
Column Position	-0.0038
Distance from Center	13742521477.0095
Focal Length	-29492912129.3269
Gradient Magnitude	-0.0041
Brightness	-0.0041

Table 1. Approximate regression coefficients for each feature.

However, some limitations remained. The segmentation occasionally missed small or hidden objects, which left some regions uncorrected. Additionally, since the segmentation mask approach is based on the objects present, the model becomes less efficient in cluttered scenes with many overlapping objects. Nonetheless, this model was certainly successful given its simple structure and minimal training. Several of the model’s predictions are shown in Figure 4.

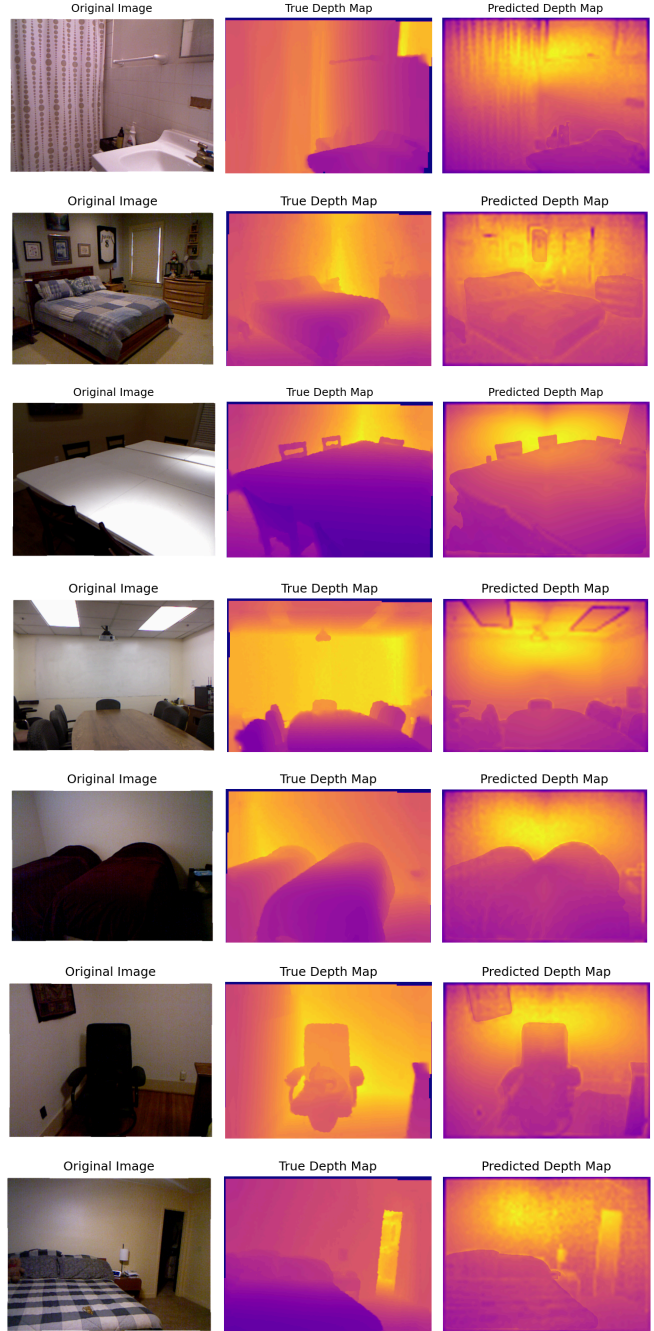
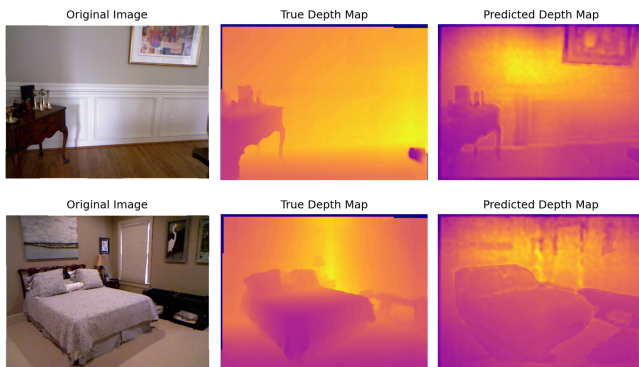


Figure 4. Nine sample predictions from the finished model.

Conclusion

This work demonstrated that regression-based monocular depth estimation can produce interpretable and reasonably accurate depth maps. On the NYU Depth V2 dataset, the model achieved an average error of approximately

0.05 m² while maintaining transparency in how predictions were made.⁵ While neural networks usually achieve higher raw accuracy, this approach offers a lightweight alternative for depth estimation in constrained settings. Future work in this area could explore adding more complex features, testing the model in other settings, or integrating the patch-based approach we used with modern neural network models. Though this area is dominated by deep learning models, our findings suggest that regression-based approaches are still useful in depth estimation.

Acknowledgements

I would like to thank Professor Michael DeWeese (UC Berkeley) for inviting me to his lab meetings and for pairing me with a mentor to help guide this project. I am especially grateful to Dhruva Karkada (UC Berkeley) for his mentorship throughout this work.

References

1. Hartley, R. I.; Sturm, P. Triangulation. *Computer Vision and Image Understanding* 1997, 68 (2), 146–157. <https://doi.org/10.1006/cviu.1997.0547>.
2. Premebida, C.; Garrote, L.; Asvadi, A.; Pedro Ribeiro, A.; Nunes, U. *High-Resolution LiDAR-Based Depth Mapping Using Bilateral Filter*. arXiv 2016, 1606.05614.
3. Yang, L.; Kang, B.; Huang, Z.; Xu, X.; Feng, J.; Zhao, H. Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data. *Thecvf.com* 2024, 10371–10381.
4. Ranftl, R.; Lasinger, K.; Hafner, D.; Schindler, K.; Koltun, V. Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 2022,

44 (3), 1623–1637.

<https://doi.org/10.1109/tpami.2020.3019967>.

5. Silberman, N.; Hoiem, D.; Kohli, P.; Fergus, R. *Indoor Segmentation and Support Inference from RGBD Images*.

https://cs.nyu.edu/~fergus/datasets/indoor_seg_support.pdf.

6. Kanopoulos, N.; Vasanthavada, N.; Baker, R. L. Design of an Image Edge Detection Filter Using the Sobel Operator. *IEEE Journal of Solid-State Circuits* 1988, 23 (2), 358–367. <https://doi.org/10.1109/4.996>.

7. Pexels.com.

<https://www.pexels.com/photo/1669799/>.

8. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; Duchesnay, É. *Scikit-learn: Machine Learning in Python; Journal of Machine Learning Research* 2011, 12 (Oct), 2825–2830.

<https://scikit-learn.org/stable/about.html#citing-scikit-learn>

9. Torchvision.

torchvision.models.detection.maskrcnn_resnet50_fpn;

https://pytorch.org/vision/stable/models.html#torchvision.models.detection.maskrcnn_resnet50_fpn.

Author

Arnav Kamath is a senior at Foothill High School who is interested in computer science, mathematics, and robotics. He hopes to pursue a career in artificial intelligence and/or robotics in the future.